

Advanced Qt 6 QML Architecture Exercise

LEVEL 2: DATA-DRIVEN MULTI-ROOM SMART HOME DASHBOARD

Objective: Graduate from basic single-property bindings to a hierarchical, data-driven architecture. You will implement an object tree structure in C++ and expose it dynamically to QML using modern Qt 6 registrations, context systems, and `QQmlListProperty`. This architecture will feed a dynamic, tab-based user interface where rooms and device interfaces scale automatically without UI-hardcoding.

1. The C++ Data Model Hierarchy

Your C++ backend must mirror a modern structural tree. Implement the three key core levels below:

Level 1: SmartHomeManager (Global Aggregator Root)

Must inherit from `QObject` and act as your single source of truth.

- **Exposed Property:** `QQmlListProperty<Room> rooms` (Read-only list property allowing QML to access and iterate over rooms).
- **Registration Strategy:** Expose this as a Singleton Instance utilizing `qmlRegisterSingletonInstance<T>()` to facilitate full compile-time optimizations and autocomplete support within QML modules.

Level 2: Room (Structural Section Model)

Must inherit from `QObject`.

- **Exposed Properties:**
 - `QString roomName` (Read-only property, e.g., "Living Room", "Kitchen").
 - `QQmlListProperty<SmartDevice> devices` (List property mapping the devices belonging strictly inside this room boundary).

Level 3: SmartDevice (Terminal Node Component)

Must inherit from `QObject`.

- **Exposed Properties:**
 - `QString deviceName` (Read-only string).
 - `DeviceEnums::Status deviceStatus` (Read-write property mapping to an external uncreatable enum structure).
 - `int batteryLevel` (Read-only property. Implement a backend mock timer loop that shifts values dynamically to ensure a functional `NOTIFY` signal loop).
- **Exposed Invocations:** Implement `Q_INVOKABLE void togglePower()` to switch device status between execution states.

- **Registration Strategy:** Register via `qmlRegisterType<SmartDevice>()` or via modern declarative macros, ensuring the engine recognizes the type properties when rendered inside data views.

2. Advanced Requirements

UNCREATABLE TYPES & ENUMS CONTEXT INJECTION

Create a standalone class or namespace called `DeviceEnums` containing a `Q_ENUM` representing state boundaries: `Off`, `On`, `Standby`, `Error`. Register this as an uncreatable namespace type using `qmlRegisterUncreatableType<T>()`.

Simultaneously, create a `SystemConfig` class containing application metadata (such as `QString homeOwnerName`) and inject the instance into the engine root using `setContextProperty()` to demonstrate context property boundaries vs. singleton types.

3. The Scalable QML UI Deliverables

Your frontend file architecture (`main.qml`) must be dynamically generated off the data models without hardcoded room arrays:

- **Multi-Tab Navigation Structure:** Use a QML `TabBar` component. Utilize a `Repeater` or `Instantiator` bound directly to the singleton array model `SmartHomeManager.rooms` to automatically populate a `TabButton` matching the string value `modelData.roomName` for every room found in the tree.
- **Dynamic Container Views:** Implement a container view (such as a `SwipeView` or a `StackLayout`) mapping its active index to the current chosen `TabBar` index.
- **Device Grid Representation:** Within each room panel view container, declare a `GridView` or `ListView` whose data provider model points directly to the active room array: `model: currentRoom.devices`.
- **Interaction Mechanics:**
 - Construct visual device "cards" showcasing the name, dynamic battery track lines, and operational state.
 - Wire up an inline button calling the invokable `modelData.togglePower()`.
 - Use the uncreatable `DeviceEnums` context safely to draw conditional design frames (e.g., if `modelData.deviceStatus === DeviceEnums.Error`, flash a warning background panel).